



DATA STRUCTURE AND ALGORITHM

Programming Practice Session

17/10/2025



2nd Session

Lists • Stacks • Queues

- **Trees & Search Trees • Hash Tables & Tries**
 - **Pointers**



Revision

1. Abstract Data Type consists 2 elements: and
2. Characteristics (order, duplicate, index access, first in, first out, last in, last out) of:

Bag:

Set:

List:

Stack:

Queue:

Revision



3. Tree (similar to linked list): root, branch, leaves, edges, parents, children, depth, height. List some types of Tree and their characteristics.

4. Hash Tables' characteristics:

5. Maps (Dictionary)' characteristics:

6. Graphs' characteristics:

(Others: sparse vectors, sparse matrices)



Revision

- 7 . Pointers: point to an ... of a variable or memory block.
- Get address of variable use operator
 - Get value at the address that the pointers point to, use:
..... operator
 - Pointer to pointer: `int ..... = &p // p is a pointer already`



Assignment 2

1. Build a singly linked list to store integers. Implement the following operations: Insert at the front Insert at the end Delete an element Search for an element

Example operations: insert front 4, insert front 6, insert back 8, insert back 10, find 5 => False, find 6 => True, delete 6, find 6 => 6, print list.

2. Given a string that only contains the characters (,), [,], {, } — check whether the string is valid (all brackets are properly opened and closed in the correct order).

Input: (())([[]{}])

Output: False

3. Create a queue that stores integers. Implement the following operations: enqueue dequeue front print Display the changes in the queue after each operation.

Example operations: enqueue 10, enqueue 20, enqueue 30, enqueue 40, dequeue, front, enqueue 50, print, dequeue, print.



Assignment 2

4. Input a sequence of n integers and insert them into a Binary Search Tree (BST). Then, input a value x and print "Found" if it exists in the tree, otherwise print "Not found"

Input: 7 55 34 87 3 99 33 21 21

Output: Found

5. Input a sequence of n integers to be inserted into an AVL Tree. After each insertion, print the height of the tree.

6. Implement insertion into a Red-Black Tree. After all insertions, print the inorder traversal of the tree along with the color of each node (R for red, B for black).

Input: 6 10 5 15 2 7 13

Output: 2(B) 5(R) 7(B) 10(B) 13(R) 15(B)

7. Build a hash table that stores integers, using chaining (linked list) to handle collisions. Implement operations such as: Insert a key Delete a key Search for a key Print the hash table contents.



Assignment 2

8. Implement a dictionary program with two main functions: 1) Store words and their meanings using a Hash Table. 2) Search suggestions by prefix using a Trie.

Input:

insert hello xin_chao

insert help giup_do

insert hero anh_hung

find hello

find hi

suggest he

Output:

hello -> xin_chao

Not found

Suggestions: help hero hello



Assignment 2

9. Implement a Binary Search Tree (BST) using pointers. Perform operations for Insertion, Search, and Traversal.

Input: 5 3 7 2 4 6 8

Output:

Inorder (sorted): 2 3 4 5 6 7 8

Enter value to search: 6

Result: Found node 6 in the tree.

10. Build a program to manage a Binary Tree with the following menu options: Add a new node, Traverse the tree (Preorder / Inorder / Postorder), Calculate the height of the tree, Find the maximum value in the tree, Delete the entire tree (free memory), Exit the program.

Submit both in-class assignments and homework. File name <student id>.cpp, zip file.